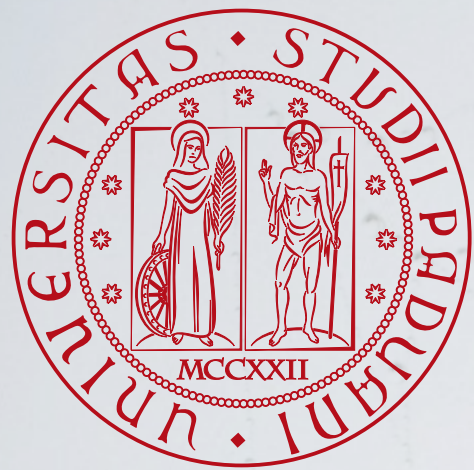




UNIVERSITÀ
DEGLI STUDI
DI PADOVA



Form Validation and AJAX

Web Applications

Master Degree in Computer Engineering

Master Degree in Cybersecurity

Master Degree in ICT for Internet and Multimedia

Academic Year 2023/2024

Nicola Ferro

Intelligent Interactive Information Access (IIIA) Hub

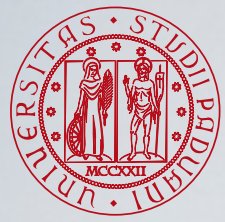
Department of Information Engineering

University of Padua



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE





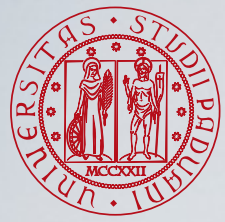
Outline



● Form Validation

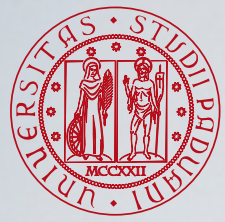
● AJAX - Scripted HTTP

Form Validation



Form Validation Example

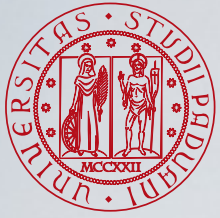
- **Form validation**: when you enter data in a Web page, the Web application checks it to see that the data is correct. If correct, the application allows the data to be submitted to the server and (usually) saved in a database; if not, it gives you an error message explaining what corrections need to be made.
- There are three main reasons to validate forms:
 - To **get the right data, in the right format**: Web applications won't work properly if the user's data is stored in the incorrect format, if they don't enter the correct information, or omit information altogether.
 - To **protect the users' accounts** by forcing them to enter secure passwords.
 - To **protect ourselves**, there are many ways that malicious users can misuse unprotected forms to damage the application they are part of.



Different Types of Form Validation

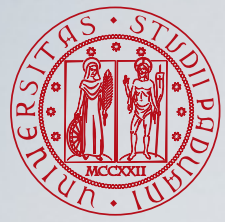


- There are two different types of form validation:
 - **Client-side validation** occurs in the browser, before the data has been submitted to the server. This is more user-friendly than server-side validation as it gives an instant response. This can be further subdivided:
 - **JavaScript validation** is coded using JavaScript. It is completely customizable.
 - Built-in **form validation** using **HTML5** form validation features. This has better performance, but it is not as customizable as JavaScript.
 - **Server-side validation** occurs on the server, after the data has been submitted. It is used to validate the data before it is saved into the database. If the data fails authentication, a response is sent back to the client to tell the user what corrections to make. Server-side validation is not as user-friendly as client-side validation, as it does not provide errors until the entire form has been submitted. However, server-side validation is the application's last line of defense against incorrect or even malicious data.
- Developers use a combination of client-side and server-side validation.



HTML 5 Validation

- One of the features of HTML5 is the ability to validate most user data without relying on scripts.
- This is done by using validation attributes on form elements, which allow you to specify rules for a form input. If the entered data follows all those rules, it is considered valid; if not, it is considered invalid.
- When an element is valid:
 - The element matches the **:valid** CSS pseudo-class; this will let you apply a specific style to valid elements.
 - If the user tries to send the data, the browser will submit the form, provided there is nothing else stopping it from doing so (e.g., JavaScript).
- When an element is invalid:
 - The element matches the **:invalid** CSS pseudo-class; this will let you apply a specific style to invalid elements.
 - If the user tries to send the data, the browser will block the form and display an error message.



Basic HTML5 Form Validation Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <meta charset="utf-8">
```

```
  <title>Form Example</title>
```

```
  <link rel="stylesheet" type="text/css"
        href="css/basic-html5-validation.css">
```

```
</head>
```

```
<body>
```

```
  <form>
```

```
    <label for="choose">In which course are you enrolled?
    Informatics or ICT?</label>
```

```
    <input id="choose" name="course" required pattern="Informatics|
    ICT|Cybersecurity">
```

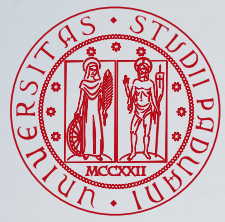
```
    <button>Submit</button>
```

```
  </form>
```

```
</body>
```

```
</html>
```

```
input:invalid {
  border: 2px dashed red;}
input:valid {
  border: 2px solid black;}
```

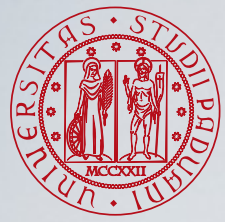


Customized Error Message

HTML5 provides the **constraint validation API** to check and customize the state of a form element. Among other things, it's possible to change the text of the error message with the **setCustomValidity()** method.

```
var email = document.getElementById("provide_email");

email.addEventListener("input", function (event) {
    if (email.validity.typeMismatch) {
        email.setCustomValidity("Please insert an email address!");
    } else {
        email.setCustomValidity("");
    }
});
```

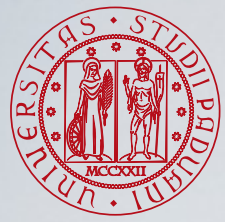


Validating Forms without a Built-in API



To validate a form, you have to ask yourself a few questions:

- What **kind of validation** should I perform? You need to determine how to validate your data: string operations, type conversion, regular expressions, etc. Remember that form data is always text and is always provided to your script as strings.
- What should I do if the form **does not validate**? You have to decide how the form will behave: should you highlight the fields which are in error? Should you display error messages?
- How can I **help the user to correct** invalid data? In order to reduce the user's frustration, it's very important to provide as much helpful information as possible in order to guide them in correcting the inputs. You should offer up-front suggestions so they know what's expected, as well as clear error messages.



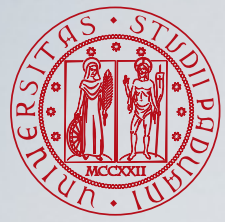
Validating Forms with Plain JavaScript



```
<div>
  <label for="provide_email">What is your e-mail?</label>
  <input type="text" id="provide_email" name="email">
  <span class="error"></span>
</div>
```

```
// Get the form element
var form = document.getElementsByTagName("form")[0];
// Get the email input element
var email = document.getElementById("provide_email");
// Get the span element next to the email element
var error = email.nextElementSibling;

// Regular expression to check whether the input is an email
address
var emailRegExp = /^[a-zA-Z0-9.!#$%&'*/+=?^_`{|}~-]+@[a-zA-Z0-9-]+
(?:\.[a-zA-Z0-9-]+)*$/;
```

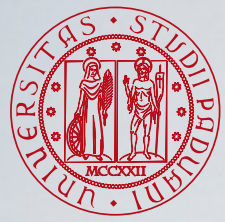


Validating Forms with Plain JavaScript



```
// This defines what happens when the user insert the email text
email.addEventListener("input", function () {
    var test = email.value.length === 0 ||
    mailRegExp.test(email.value);

    if (test) {
        email.className = "valid";
        error.innerHTML = "";
    } else {
        email.className = "invalid";
        error.innerHTML = "Please insert an e-mail address";
        error.className = "error";
    }
});
```



Validating Forms with Plain JavaScript

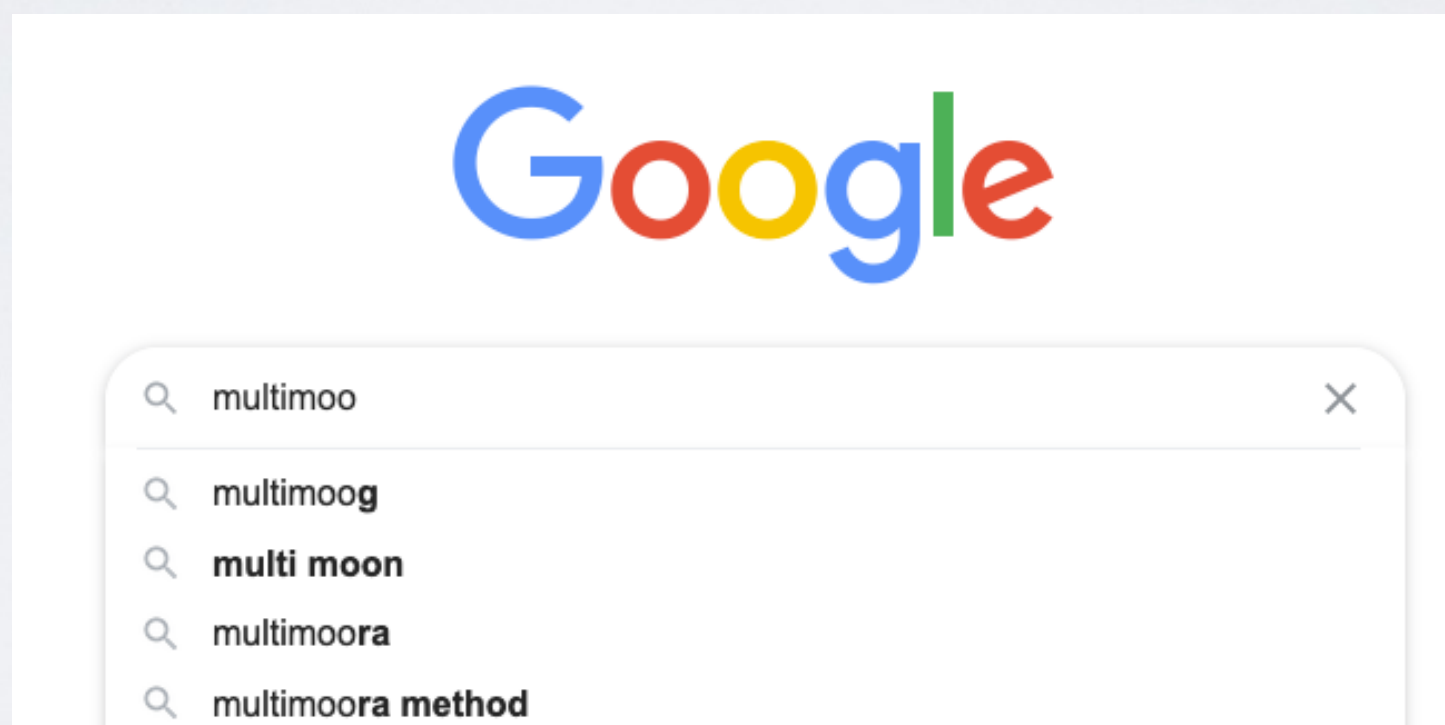


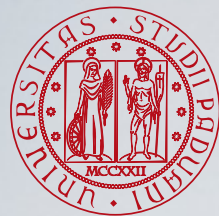
```
// This defines what happens when the user tries to submit the data
form.addEventListener("submit", function (event) {
    var test = email.value.length === 0 ||
    emailRegExp.test(email.value);

    if (test) {
        email.className = "valid";
        error.innerHTML = "";
    } else {
        email.className = "invalid";
        error.innerHTML = "I expect an e-mail!";
        error.className = "error active";
        event.preventDefault();
    }
});
```

AJAX - Scripted HTTP

- You may have seen AJAX used on many website, even if you were not aware that it was being used.
- Live search (or autocomplete) commonly uses AJAX. When you type into the search bar of the home page, sometimes the results come up before you have finished typing (e.g. Google)

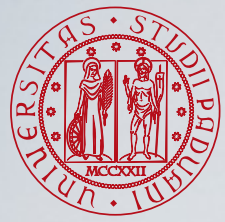




AJAX Examples



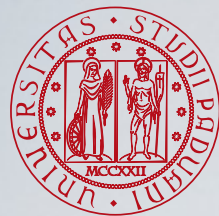
- Websites with user-generated content (Twitter, Facebook etc.) allow you to display information (e.g. latest photograph) on your own page. This involves collecting data from their servers.
- When you are shopping online and add items to your shopping cart, it is updated without you leaving the page, and the site may display a confirmation message
- When you are registering in a website, a script may check whether your username is available before you have complete the rest of the form



Synchronous vs Asynchronous



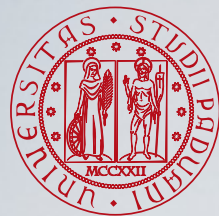
- When a browser comes across a `<script>` tag, it will typically stop processing the rest of the page until it has loaded and processed it. This is an example of **synchronous processing model**
- This can need time, e.g. if for example the script requires data from the server. Then you need to further wait the answer from the server
- AJAX instead uses an **asynchronous (non-blocking) processing model**, i.e. the user can do other things while the web browser is waiting for the data to load, speeding up the user experience.
- When the server responds with the data, an event is fired, which can call a function that processes the data. This function can update only one element of the page, instead of the whole page



AJAX



- Historically, **AJAX** stands for **Asynchronous JavaScript And XML**, an acronym containing the technologies used at the time (JavaScript and XML). AJAX now indicates a group of technologies that offer asynchronous functionality in the browser.
- The key feature of an Ajax application is that it uses scripted HTTP to initiate data exchange with a web server without causing pages to reload.
- AJAX uses the **XMLHttpRequest** object to communicate with servers. It can send and receive information in various formats, including JSON, XML, HTML, and text files. AJAX's most appealing characteristic is its "asynchronous" nature, which means it can communicate with the server, exchange data, and update the page without having to refresh the page.
- The ability to **avoid page reloads** results in responsive web applications.
- A web application might use Ajax technologies to log user interaction data to the server or to improve its start-up time by displaying only a simple page at first and then downloading additional data and page components on an as-needed basis.

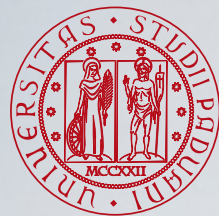


Using XMLHttpRequest

- Browsers define their HTTP API on the `XMLHttpRequest` class. Each instance of this class represents a single request/response pair, and the properties and methods of the object allow you to specify request details and extract response data.

```
var request = new XMLHttpRequest();
```

- An HTTP request consists of four parts:
 - the HTTP request method;
 - the URL being requested;
 - an optional set of request headers, which may include authentication information;
 - an optional request body.
- The HTTP response sent by a server has three parts:
 - a numeric and textual status code that indicates the success or failure of the request;
 - a set of response headers;
 - the response body.



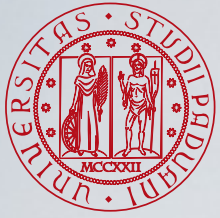
Specifying the Request



- After creating an XMLHttpRequest object, the next step in making an HTTP request is to call the **open ()** method of your XMLHttpRequest object:

```
request.open( 'GET' , 'http://www.example.org/some.file' );
```

- The first parameter of the **open ()** method is the HTTP **request method**. Keep the method all-capitals as per the HTTP standard, otherwise some browsers might not process the request.
- The second parameter is the **URL** that is the subject of the request. This is relative to the URL of the document that contains the script that is calling **open ()**. As a security feature, you cannot call URLs on third-party domains. Be sure to use the exact domain name on all of your pages or you will get a "permission denied" error.



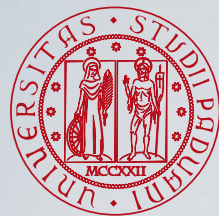
Specifying the Request

- To set the request headers, if any, you can use the `setRequestHeader( )` method.
- POST requests, for example, need a “Content-Type” header to specify the MIME type of the request body:

```
request.setRequestHeader( "Content-Type", "text/plain" );
```

- If you call `setRequestHeader()` multiple times for the same header, the new value does not replace the previously specified value: instead, the HTTP request will include multiple copies of the header or the header will specify multiple values.
- The final step in making an HTTP request with `XMLHttpRequest` is to send it off to the server, with the `send( )` method:

```
request.send( );
```



Encoding the Request Body



- Recall: HTTP POST requests include a request body that contains data the client is passing to the server.
- **Form-encoded requests:** URI encoding (replacing special characters with hexadecimal escape codes) on the name and value of each form element, separate the encoded name and value with an equals sign, and separate these name/value pairs with ampersands.

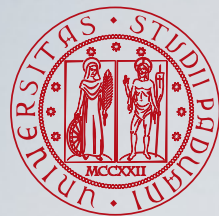
```
find=pizza&zipcode=02134&radius=1km
```

- form data encoding format has a formal MIME type

```
application/x-www-form-urlencoded
```

- **Json-encoded requests:**

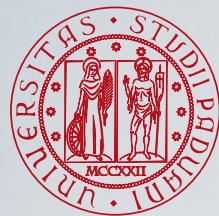
```
request.setRequestHeader("Content-Type", "application/json");
```



Cross Origin Resource Sharing



- The XMLHttpRequest object can normally issue HTTP requests only to the server from which the document that uses it was downloaded, (same-origin security policy). That is, browsers do not load AJAX responses from other domains. There are different workarounds, among those we present CORS.
- **Cross-Origin Resource Sharing**, (**CORS**): is a mechanism that uses additional HTTP headers to let a user agent gain permission to access selected resources from a server on a different origin (domain) than the site currently in use. A user agent makes a cross-origin HTTP request when it requests a resource from a different domain, protocol, or port than the one from which the current document originated.
- Example: A HTML page served from `http://domain-a.com` makes an `<img> src` request for `http://domain-b.com/image.jpg`.
- The Cross-Origin Resource Sharing standard works by adding **new HTTP headers** that allow servers to describe the set of origins that are permitted to read that information using a web browser.
- Cross-origin requests do not normally include any user credentials: username and password, cookies, HTTP authentication tokens, ...

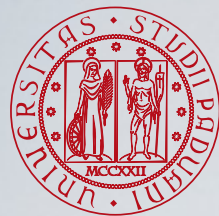


Retrieving the Response

- The same object that sent the request deals with the answer. When you send the request, you should provide the name of a JavaScript function to handle the response:

```
request.onload = nameOfTheFunction;
```

- The function needs to check the request's state. If the state has the value of `XMLHttpRequest.DONE`, that means that the full server response was received and it's OK to continue processing it.
- The full list of the `readyState` values is as follows:
 - 0 (uninitialized) or (request not initialized), `open ( )` has not been called yet;
 - 1 (loading) or (server connection established), `open ( )` has been called;
 - 2 (loaded) or (request received), headers have been received;
 - 3 (interactive) or (processing request), the response body is being received;
 - 4 (complete) or (request finished and response is ready), the response is complete.

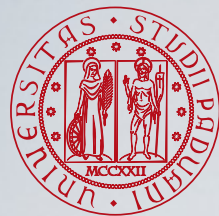


Retrieving the Response

- Next, check the response code of the HTTP response (successful = 200).

```
request.status == 200
```

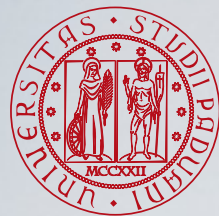
- After checking the state of the request and the HTTP status code of the response, you have two options to access that data:
 - **request.responseText** – returns the server response as a string of text;
 - **request.responseXML** – returns the response as an XMLDocument object you can traverse with JavaScript DOM functions.



Example



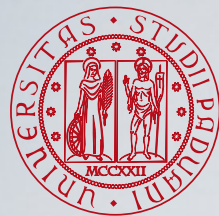
```
(function() {  
    var httpRequest;  
    document.getElementById( 'ajaxButton' ).addEventListener( 'click',  
        makeRequest );  
  
    function makeRequest() {  
        httpRequest = new XMLHttpRequest();  
  
        if ( !httpRequest ) {  
            alert( 'Giving up :( Cannot create an XMLHTTP instance' );  
            return false;  
        }  
        httpRequest.onload = alertContents;  
        httpRequest.open( 'GET', 'test.html' );  
        httpRequest.send();  
    }  
})
```



Example



```
function alertContents() {  
    if (httpRequest.readyState === XMLHttpRequest.DONE) {  
        if (httpRequest.status === 200) {  
            alert(httpRequest.responseText);  
        } else {  
            alert('There was a problem with the request.');        }  
    }  
}  
})();
```



Types of receivable data

● HTML

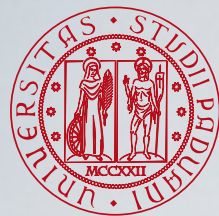
- Pros: easy to write, request and display; The data sent from the server can go straight into the page, no need to process it (e.g. through JavaScript).
- Cons: the server must produce the HTML in a format that is ready for use on our page; it is not well-suited for use in applications other than web browsers, i.e. **no good data portability**.

● XML

- Pros: it is a flexible data format, and can represent complex structures. It works well with different platforms and applications. It is processed using the same HTML DOM methods.
- Cons: it is considered a verbose language, the tags add a lot of extra characters to the file being sent; it can require a lot of code to be processed.

● JSON

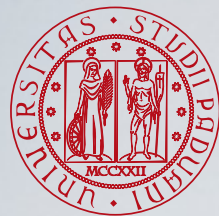
- Pros: it can be called from any domain (CORS); more concise than the other twos; commonly used with JavaScript (it has gained wide use across web applications).
- Cons: the syntax is very strict (unlike HTML), i.e. a missed quote, comma or colon can “break” the file; since it is still JavaScript, it can contain malicious content, therefore only use JSON that has been produced by trusted sources



Loading JSON with AJAX



- The server sends JSON data to a web browser as a **string**
- When it reaches the browser, a script must convert the string into a JavaScript object — **deserialization of the object** — through the `parse( )` method of the `JSON` object. It is a global object, so you do not need to instantiate it.
- Once the string has been parsed, the script can access the data in the object, and use it to create HTML.
- The HTML is added to the page using the `innerHTML` property, thus only use it when you are confident that it does not contain malicious code.
- The method `JSON.stringify( )` converts objects into a string using JSON notation, thus to send the object from the browser back to the server, a.k.a. **serialization of the object**.



Example

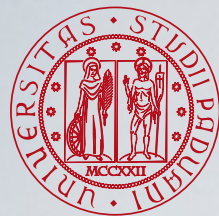


```
var xhr =new XMLHttpRequest();

xhr.onload = function() {
    if(xhr.status === 200) {
        // responseText is a property containing the response from the server
        responseObject = JSON.parse(xhr.responseText);
        var newContent = '';
        for (var i = 0; i < responseObject.events.length; i++) {
            newContent += '<div class="event">';
            newContent += '<img src="" + responseObject.events[i].map +""';
            newContent += 'alt="" + responseObject.events[i].location + ""/ >' ;
            newContent += '<p><b>' + responseObject.events[i].location + '</b><br>';
            newContent += responseObject.events[i].date + '</p>' ;
            newContent += '</div>';
        }

        //Update the page
        document.getElementById('content').innerHTML = newContent;
    }
}
```

...



Example



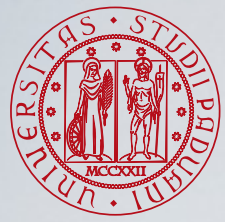
...

```
xhr.open( 'GET', 'data/data.json' );
```

```
xhr.send( );
```

Where data.json is a file of the type:

```
{  
  "events": [  
    { "location": "San Francisco, CA", "date": "May 1", "map": "img/map-ca.pnh" },  
    ...  
  ]  
}
```



Fetch



- More recently Javascript has introduced a new way of sending requests to servers and send data through the method **fetch**.
- The method is not supported by older browsers, so be careful.
- Basic syntax:

```
var promise = fetch(url, [options])
```

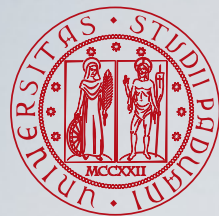
- Url: the url to be reached
- Options: corresponds to optional parameters, like methods, headers, etc.

- The JavaScript Fetch object is based on the use of a **Promise**, an **object** that encapsulates the result of an **asynchronous operation**.
 - The key idea behind a Promise is that, when it resolves (i.e., when the answer is returned from the server) it becomes an object of type **Response**, that presents useful methods and properties
- Invoked without options, the syntax of the previous slide corresponds to a GET call that downloads the content of url.
- The act of obtaining an answer is typically performed in 2 steps:
 - Check the status (if everything went right with the server)
 - Work with the body of the answer

```
let response = await fetch(url);

if (response.ok) { // if HTTP-status is 200-299
  //receive the body of the answer
  let json = await response.json();
} else {
  alert("HTTP-Error: " + response.status);
}
```

- Here fetch is used to GET information from a URL. The keyword await is used (in the context of Promise objects) to wait for the fulfilled value of the promise, in this case the response of the server.
- The method json() converts the body of the response in its json format



Further Readings

- MDN Web Docs: Resources for Developers, by Developers. <https://developer.mozilla.org/en-US/>
- Duckett, J., Ruppert, G., and Moore, J. (2014). JavaScript & jQuery: Interactive Front-end Web Development. Wiley.
- Flanagan, D. (2011). JavaScript: the Definitive Guide. O'Reilly Media, Inc.